Practical:06	Implementation of chain matrix multiplication using dynamic programming.
__/__ /2026	

Objective:

- To implement the Matrix Chain Multiplication problem using Dynamic Programming.
- To find the minimum number of scalar multiplications required to multiply a chain of matrices.
- To understand how Dynamic Programming avoids repeated calculations.

Program:

```
#include <iostream> #include
<climits> using namespace
std;

// Function to find minimum multiplication cost void matrixChainMultiplication(int
p[], int n)
{    int
m[n][n];

   // Cost is 0 when multiplying one matrix
for (int i = 1; i < n; i++)        m[i][i] = 0;

   // length is the chain length     for (int length
= 2; length < n; length++)
   {        for (int i = 1; i < n - length + 1;
i++)
      {           int j = i +
length - 1;           m[i][j] =
INT_MAX;

         for (int k = i; k < j; k++)
         {
```

```
                int cost = m[i][k] +                    m[k
+ 1][j] +                    p[i -
1] * p[k] * p[j];
                if (cost < m[i][j])
m[i][j] = cost;
            }
        }
    }

    cout << "Minimum number of scalar multiplications: "
         << m[1][n - 1] << endl;
}

int main()
{    int n;

    cout << "Enter number of matrices: ";
    cin >> n;

    int p[n + 1];

    cout << "Enter dimensions of matrices:\n";

    cout << "For matrices A1, A2, ..., An, enter "
         << n + 1 << " dimensions:\n";

    for (int i = 0; i <= n; i++)        cin
>> p[i];

    matrixChainMultiplication(p, n + 1);

    return 0;
```

}

Output:

```
Enter number of matrices: 4
Enter dimensions of matrices:
For matrices A1, A2, ..., An, enter 5 dimensions:
10 20 30 40 50
Minimum number of scalar multiplications: 38000
```

Analysis:

- Dynamic Programming avoids recalculating the same subproblems.
- All possible parenthesizations are considered through the split point k.
- The algorithm finds the optimal multiplication order, not the actual multiplication of matrices.
- It is more efficient than checking every possible parenthesization separately.

For a chain of n matrices, the algorithm uses a DP table $C(i, j)$, where $C(i, j)$stores the minimum multiplication cost for matrices $A_i$through A_j.

Time Complexity

$$O(n^3)$$

There are approximately $O(n^2)$entries in the DP table, and for each entry, all possible split positions are checked, giving $O(n)$work.

Space Complexity

$$O(n^2)$$

The $n \times n$DP table is required to store the minimum costs.

Conclusion:

The Matrix Chain Multiplication problem was successfully implemented using Dynamic Programming. The program determines the optimal order of matrix multiplication that requires the minimum number of scalar multiplications. This method reduces unnecessary calculations by storing previously calculated results.

Practical:07	Implementation of making a change problem using dynamic programming.
__/__ /2026	

Objective:

- To implement the Coin Change/Making Change problem using Dynamic Programming.
- To find the minimum number of coins required to make a given amount.
- To understand how Dynamic Programming provides an efficient solution by storing previously calculated results.

Program:

```
#include <iostream>
#include <vector> #include <climits> using
namespace std; int main() {     int n, amount;     cout
<< "Enter number of coin denominations: ";
    cin >> n;

    vector<int> coins(n);

    cout << "Enter coin denominations: ";
for (int i = 0; i < n; i++)         cin >> coins[i];

    cout << "Enter amount: ";     cin
>> amount;

    // dp[i] = minimum coins required to make amount i
vector<int> dp(amount + 1, INT_MAX);

    // 0 coins are required to make amount 0
dp[0] = 0;

    // Calculate minimum coins for each amount      for
(int i = 1; i <= amount; i++)
```

```
    {
        for (int coin : coins)
        {
            if (coin <= i && dp[i - coin] != INT_MAX)
            {
                dp[i] = min(dp[i], dp[i - coin] + 1);
            }
        }
    }

    if (dp[amount] == INT_MAX)
    {
        cout << "Change cannot be made for the given amount." << endl;
    }
else
    {
        cout << "Minimum number of coins required: "
             << dp[amount] << endl;
    }
return 0;  }
```

Output:

```
Enter number of coin denominations: 4
Enter coin denominations: 1 2 5 10
Enter amount: 18
Minimum number of coins required: 4
```

Analysis:

- Dynamic Programming stores solutions of smaller amounts and uses them to solve larger amounts.
- It avoids repeated calculations of the same amounts.
- The algorithm finds the minimum number of coins required.
- If the amount cannot be formed using the given denominations, the program reports that change cannot be made

The Making Change problem is solved using Dynamic Programming by finding the minimum number of coins required to make every amount from 0to the target amount.

The DP array $C(j)$stores the minimum number of coins required to form amount j. The recurrence is: $C(j) = \min\{C(j - d_i) + 1\}$

$d_i \leq j$

where $d_i$represents a coin denomination.

Time Complexity

If there are n coin denominations and the target amount is V:

$$\boxed{O(nV)}$$

For every amount from 1to V, the algorithm checks all $n$coin denominations. Space Complexity

$$\boxed{O(V)}$$

An array of size $V + 1$is used to store the minimum number of coins for each amount.

Conclusion:

The Making Change problem was successfully implemented using Dynamic Programming. The program calculates the minimum number of coins required to form a given amount. By storing the solutions of smaller amounts, the algorithm avoids repeated calculations and provides an efficient solution to the problem.